Java SE 17 Programmer II（1Z0-826-JPN）　試験内容 1 コレクションとジェネリクス　#02

# ジェネリクス

型をパラメータにして、まちがいをコンパイル時に見つける

各ページの下に、根拠にした資料（Oracle の試験内容チェックリスト、Java 言語仕様 JLS、The Java Tutorials の Generics）を書いた。コードの結果は JDK 17（Eclipse Temurin 17.0.21）で実行して確かめた

# 全体像

**1 宣言と使い方**
ジェネリッククラス・メソッド、型引数、ダイヤモンド、型推論

**2 境界とワイルドカード**
extends・&・? extends・? super・?、PECS、不変性

**3 イレイジャと制限**
型消去、原型、new T()・static T・instanceof・配列ができない理由

```
List<String> list = new ArrayList<>();
list.add("a");
String s = list.get(0);  // キャスト不要
list.add(1);             // コンパイルエラー
```

ジェネリクスの価値は「型のまちがいを実行前に止める」こと。実行時には型引数は消えている（イレイジャ）。

出典：Oracle 試験内容チェックリスト 1「総称型の使用」／The Java Tutorials: Generics「Why Use Generics?」

第1章

# 宣言と使い方

型パラメータを宣言し、使うときに型引数を当てはめる。

# ジェネリッククラスとジェネリックメソッド

```
class Box<T> {            // T：型パラメータ
  private T value;
  Box(T value) { this.value = value; }
  T get() { return value; }
}
Box<String> b = new Box<>("hi"); // String：型引数
```

慣習の名前：T 型・E 要素・K キー・V 値・R 結果。型引数は参照型だけ。

```
static <T> List<T> twice(T t) {
  return List.of(t, t);
}
Main.<String>twice("x"); // 明示はメソッド名の前
twice(5);                // T = Integer と推論
```

| | |
|---|---|
| `var l = new ArrayList<>();` | ArrayList<Object> |
| `new ArrayList<?>()` | エラー |
| `show()`（T... 引数なし） | Object[0] |

出典：JLS 8.1.2・8.4.4・15.9.1・15.12.4.2／JDK 17 での実行結果

# ジェネリック型を実装する

```
interface Animal<T> { T speak(T t); }
class Dog implements Animal<String> {
  public String speak(String s) {   // ○
    return s + s;
  }
}
```

型引数を当てはめた形 speak(String) で書く。speak(Object) との橋渡しはコンパイラのブリッジメソッド。

```
class Dog implements Animal<String> {
  public Object speak(Object s) { ... } // ×
}
```

speak(String) を実装していない＋同じイレイジャで衝突（name clash）の２つのエラー。

```
record Pair<K, V>(K first, V second) {
  Pair<V, K> swap() { return new Pair<>(second, first); }
}
```

出典：JLS 8.4.8・8.10、The Java Tutorials「Effects of Type Erasure and Bridge Methods」／JDK 17 でのコンパイル結果

第2章

# 境界とワイルドカード

読むなら extends、入れるなら super。

# 不変性と、配列の共変性

Integer は子 → Number

List<Integer> × 子ではない List<Number>

```
List<Number> ln = li;          // エラー
List<? extends Number> l = li; // ○
List<?> l2 = li;               // ○
```

**配列は共変（実行時に確かめる）**

```
Object[] arr = new String[1]; // コンパイル○
arr[0] = 1;  // ArrayStoreException
```

> もし List<Integer> を List<Number> として扱えたら、Double を入れられてしまう。ジェネリクスはそれをコンパイル時に止める。

出典：JLS 4.10.2・10.5／The Java Tutorials「Generics, Inheritance, and Subtypes」／JDK 17 での実行結果

# ワイルドカードでできること

| 型 | add できるもの | get の型 | 代入できる List |
|---|---|---|---|
| List<Number> | Number とその子 | Number | List<Number> だけ |
| List<? extends Number> | **null だけ** | Number | List<Integer>・List<Double>… |
| List<? super Integer> | Integer とその子 | Object | List<Integer>・<Number>・<Object> |
| List<?> | null だけ | Object | どの List でも |

```
static void copy(List<? super Number> dst,
                 List<? extends Number> src) {
  dst.addAll(src);
}
```

**PECS**：Producer Extends, Consumer Super。値を出す引数は extends、値を入れる引数は super。両方するなら List<T>。

出典：JLS 4.5.1・5.1.10、The Java Tutorials「Wildcards」「Guidelines for Wildcard Use」／JDK 17 でのコンパイル結果

# 境界のある型パラメータ

```
static <T extends Comparable<T>> T max(List<T> list) {
  T best = list.get(0);
  for (T t : list)
    if (t.compareTo(best) > 0) best = t;
  return best;
}
max(List.of("pear", "apple", "fig")) // pear
```

境界の型のメソッド（compareTo・doubleValue）を T に呼べる。

| 宣言 | 結果 |
| --- | --- |
| `<T extends Number & Comparable<T>>` | ○ |
| `<T extends Comparable<T> & Number>` | × クラスは先頭 |
| `<T implements Comparable<T>>` | × extends で書く |
| `Shelf<String>`（T extends Number） | × not within bounds |

出典：JLS 4.4 Type Variables・4.5／JDK 17 でのコンパイルと実行の結果

第3章

# イレイジャと制限

実行時に型引数は残らない。だからできないことがある。

# 型消去（イレイジャ）

**List<String>**
コンパイル時 → **List**
実行時（型引数は消える）

**T extends Number** → **Number**
境界の型に

```
new ArrayList<String>().getClass()
  == new ArrayList<Integer>().getClass() // true
```

### 原型（raw type）とヒープ汚染

```
List raw = new ArrayList();
raw.add("x"); raw.add(1);
List<String> strs = raw;  // 警告だけ
strs.size();              // 2
String s = strs.get(1);   // ClassCastException
```

> エラーは入れたときでなく、取り出したときに出る。

出典：JLS 4.6 Type Erasure・4.8 Raw Types・4.12.2／JDK 17 での実行結果

# ジェネリクスでできないこと

| コード | JDK 17 のエラー |
|---|---|
| `new T()` | unexpected type（required: class） |
| `static T shared;` | non-static type variable T cannot be referenced from a static context |
| `o instanceof T`／Object に `instanceof List<String>` | Object cannot be safely cast to … |
| `new List<String>[3]` | generic array creation |
| `print(List<String>)` と `print(List<Integer>)` | name clash: … have the same erasure |

できるもの：`new List<?>[3]`、`o instanceof List<?>`、`(T[]) new Object[3]`（unchecked の警告）。

推論：`var x = pick("a", 1)` は通るが、`String s = pick("a", 1)` はエラー。

出典：The Java Tutorials「Restrictions on Generics」／JLS 15.10.1・15.20.2・8.4.2／JDK 17 でのコンパイル結果